

Redspher

Full Stack Developer

Mobil interjúfelkészítő PDF

Cél

Gyors, telefonról is jól olvasható felkészülés a Redspher Full Stack Developer interjúra. A hangsúly: React + TypeScript, Node.js/backend, REST, AWS, CI/CD, business-driven delivery és AI/agent/MCP irány.

Fontos megjegyzés

Ez az anyag a megadott job description alapján készült. Nem tartalmaz kitalált, korábban elhangzott interjúkérdéseket. A 'valószínű kérdések' részek szakmai becslések hasonló full-stack interjúk alapján.

Ajánlott használat: először a gyors összefoglalót olvasd át, utána menj végig a Q&A kártyákon, végül gyakorold hangosan az angol válaszvázakat.

1. A pozíció lényege

Mit kell belőled látnia az interjúztatónak.

- Full-stack fejlesztő, aki end-to-end feature-ökben gondolkodik, nem csak frontend ticketekben.
- Erős React + TypeScript alapok: komponensek, state, adatok, UI minőség, karbantarthatóság.
- Backend oldalon Node.js / service gondolkodás: REST API, validáció, hibakezelés, auth, integrációk.
- AWS környezetben való munkavégzés: nem feltétlen cloud architect szint, de értsd a deploy, logs, security, networking és managed services alapjait.
- Business-driven pragmatizmus: logisztikai / pricing / operations problémákból kell működő terméket csinálni.
- AI/automation kíváncsiság: LLM-ek, agentek, MCP, workflow automation, prototípus szemlélet.
- Plusz pont: PHP legacy rendszerek megértése vagy integrálása.

Egy mondatos pozicionálás

I am a full-stack developer with strong React and TypeScript experience, solid Node.js/backend exposure, and a pragmatic mindset: I like turning business problems into reliable, maintainable features. I am also actively interested in AI-assisted workflows and automation.

Mit hangsúlyozz a saját tapasztalatodból?

React/TypeScript SPA tapasztalat, Node.js tooling és backend/realtime rendszerek, REST/GraphQL integrációk, CI/CD/deployment együttműködés, monitoring/logging, plusz AI-alapú vagy automatizációs hobby/projekt tapasztalat. Ezek jól illeszkednek a JD-hez.

2. 60 perces gyors felkészülési terv

Ha kevés időd van, ezekre menj rá.

0-10 perc: olvasd át a pozíció lényegét és a pitch-et.

10-25 perc: React + TypeScript kérdések.

25-40 perc: Node.js, REST, hibakezelés, API design.

40-50 perc: AWS + CI/CD alapok.

50-60 perc: AI/MCP és business/pragmatic kérdések.

Interjúban ne csak technológiát sorolj

AJD business-oriented csapatot ír. Minden technikai válasznál mondd el: miért ez a megoldás, milyen trade-off van, hogyan mérnéd, és hogyan kommunikálnád product/business stakeholdernek.

Három fő üzenet, amit újra és újra visszahozhatsz:

- I build maintainable end-to-end features, not just isolated components.
- I care about reliability, observability, and pragmatic trade-offs.
- I am comfortable learning new AI/automation patterns and turning them into practical prototypes.

3. Rövid angol bemutatkozás

Ezt érdemes hangosan gyakorolni.

Default verzió

I am a full-stack developer with a strong frontend background in React and TypeScript, and solid experience around Node.js, APIs, automation, and production-oriented delivery. In my previous roles I worked closely with product, QA, and business stakeholders, so I am comfortable translating requirements into practical technical solutions. What attracts me to this role is the combination of logistics business problems, modern full-stack development, AWS, and the opportunity to explore AI-driven workflows in a pragmatic way.

Ha rövidebben kell

I am a pragmatic full-stack developer with strong React and TypeScript experience, good backend exposure with Node.js and APIs, and a strong interest in automation and AI-assisted workflows. I like roles where technology is connected to real business impact.

Miért Redspher?

What I like about Redspher is that the role is not only about coding screens or APIs. It is about improving core logistics applications and business workflows. The AI and automation angle is also interesting to me because I enjoy exploring tools that can make internal processes faster and more reliable.

4. React + TypeScript fókusz

Itt valószínűleg erősnek kell tűnnöd.

REACT

How do you structure a complex React feature?

I usually separate UI components, state/data fetching, domain logic, and API access. I try to keep reusable components small, keep side effects in hooks or service layers, and make the data flow easy to follow. For a business-critical feature, I would also think about loading states, error states, analytics, and testability.

TYPESCRIPT

What makes TypeScript useful in a React project?

TypeScript helps catch integration mistakes early: wrong props, wrong API shapes, missing fields, invalid states. I especially like using typed API DTOs, discriminated unions for UI states, and strict props for reusable components. It is not a replacement for tests, but it reduces many simple runtime mistakes.

PRAGMATIC

How do you avoid over-engineering React code?

I start with the simplest structure that makes the feature readable. I extract reusable components only when the duplication is real or the abstraction makes the business concept clearer. I also avoid global state unless data is truly shared across multiple parts of the application.

5. React kérdések - gyors válaszok

Controlled vs uncontrolled components?

Controlled: React state is the source of truth. Good for validation and dynamic UI. Uncontrolled: DOM keeps the state, useful for simple forms or performance-sensitive cases. In business apps I usually prefer controlled or form-library-managed inputs for predictability.

When would you use memoization?

When a component does expensive computation or receives stable props but re-renders often. I would not add `useMemo`/`useCallback` everywhere, because it adds complexity. I would first identify a real rendering problem with profiling or obvious code structure.

How do you handle API loading and errors?

I model loading, success, empty and error states explicitly. For retryable errors I show a clear retry action; for validation errors I map backend messages to fields; for unexpected failures I log enough context for debugging.

How do you design reusable components?

Reusable components should have a small API, predictable behavior, accessible markup, and avoid knowing too much about business-specific data. I prefer composition over too many boolean props.

How do you keep frontend and backend contracts aligned?

Typed DTOs, schema validation where useful, generated clients if the project supports it, and API contract reviews. In practice I also pay attention to backward-compatible API changes.

6. Node.js / backend fókusz

Mutasd, hogy nem csak UI fejlesztő vagy.

How would you design a REST endpoint for a logistics feature?

I would start from the business operation: what resource is being read or changed, who is allowed to do it, and what consistency is required. Then I would define clear request/response contracts, validation, error codes, idempotency if the operation can be retried, and logging/metrics for production support.

How do you handle errors in a Node.js API?

I separate expected business errors from unexpected technical errors. Expected errors get clear status codes and safe messages. Unexpected errors are logged with correlation IDs and return a generic response. I also avoid swallowing async errors and make sure the process behavior is safe under failure.

What should be validated on the backend even if the frontend validates it?

Everything that affects correctness or security: required fields, types, ranges, authorization, IDs, business rules, and payload size. Frontend validation is for user experience; backend validation is for system integrity.

How do you approach performance in Node.js?

Avoid blocking the event loop, paginate large results, use efficient database queries, cache carefully, and add observability before guessing. For CPU-heavy work, move it to workers, queues, or a separate service.

7. REST API design

Gyakori full-stack interjúterület.

What is a good REST API?

A good REST API has clear resources, predictable HTTP methods, correct status codes, stable contracts, good validation, useful errors, and documentation. It should be easy for a frontend developer and backend developer to reason about the same business operation.

How do you version an API?

For breaking changes, I prefer explicit versioning, for example /v1 or versioned routes, depending on team standards. For non-breaking changes, such as adding optional response fields, I usually keep the same version. The key is to avoid breaking existing consumers unexpectedly.

Is adding a new enum value breaking?

It can be breaking for clients that assume the enum is closed and do not have a fallback. Technically it may look additive, but in real systems it can break UI logic, switch statements, translations, or analytics. I would communicate it and make clients resilient to unknown values.

How do you design error responses?

Use consistent structure: code, message, details if safe, and correlation ID. The frontend should be able to distinguish validation errors, auth errors, not found, conflict, and unexpected errors without parsing random text.

When is idempotency important?

For operations that may be retried: payments, booking, shipment creation, order changes, or any workflow where duplicate requests could create duplicate business effects. Use idempotency keys or natural unique constraints depending on the case.

8. AWS alapok full-stack szemmel

Nem cloud architect vizsga, hanem production awareness.

- Know where code runs: Lambda, ECS, EC2, containerized services, or managed runtime.
- Know where data lives: RDS/PostgreSQL, DynamoDB, S3, caches, queues.
- Know how traffic enters: API Gateway, ALB, CloudFront, Route 53.
- Know observability: CloudWatch logs/metrics, alarms, tracing if enabled.
- Know security basics: IAM least privilege, secrets management, VPC/security groups, encrypted storage.
- Know deployment basics: CI/CD pipeline, environment variables, staging/prod, rollback.

What AWS services would you expect in a full-stack web app?

A typical setup could use CloudFront/S3 for static assets, API Gateway or ALB for APIs, Lambda/ECS/EC2 for backend services, RDS or DynamoDB for data, SQS for async work, CloudWatch for logs and metrics, and IAM/Secrets Manager for security.

How do you debug a production issue in AWS?

Start with user impact and timestamps, then check logs, metrics, deployment history, recent config changes, and downstream dependencies. I would use correlation IDs across frontend and backend when possible, and avoid changing multiple things at once during investigation.

What does least privilege mean in IAM?

A service or user should only get the permissions needed for its task, and nothing more. For example, a Lambda that reads one table should not get full account-wide DynamoDB access. It limits blast radius if something goes wrong.

9. Git, CI/CD és release szemlélet

What do you expect from a good CI/CD pipeline?

It should build the application, run tests and checks, produce a reproducible artifact, deploy consistently, and make rollback possible. It should also separate environments clearly and expose enough logs to diagnose failed deployments.

How do you reduce deployment risk?

Small changes, feature flags when useful, staging validation, automated checks, monitoring after release, rollback plan, and communication with QA/product. For risky backend changes I also think about backward compatibility and database migrations.

How do you use Git in a team?

Small focused branches, clear commit messages, pull requests, code review, and keeping the main branch releasable. I prefer discussing architectural concerns early instead of hiding them in a huge PR.

Saját példát ide érdemes kötni

Beszélhetsz arról, hogy dolgoztál staging/prod release folyamatban, QA-val és business/product oldallal együtt, illetve monitoring/logging alapján kerestél hibákat. Ez nagyon jól illik a business-oriented full-stack szerephez.

10. Architecture + pragmatic mindset

AJD szerint részt veszel technikai döntésekben.

How do you make architectural decisions?

I start from constraints: business goal, expected traffic, data consistency, team skills, deadline, operations, and cost. Then I compare options and choose the simplest design that can grow. I like documenting trade-offs, not just the final decision.

How do you translate business needs into technical work?

I clarify the workflow, edge cases, user roles, data ownership, and success metric. Then I split the solution into frontend, backend, data, and release tasks. I try to confirm assumptions early with product or business stakeholders.

How do you avoid building the wrong thing?

Ask clarifying questions, prototype risky parts, show early increments, validate with stakeholders, and keep the solution observable. In business systems, misunderstanding the workflow is often more dangerous than picking the wrong library.

How do you decide between quick fix and proper refactor?

I consider user impact, risk, how often the code changes, and whether the quick fix increases future cost. Sometimes a safe tactical fix is right, but I would make technical debt visible and propose a follow-up when it matters.

11. AI, LLM, MCP és agent alapok

A JD-ben ez kiemelt innovációs irány.

Egyszerű magyarázat

LLM: nyelvi modell, amely szöveg alapján következtet és generál.
Agent: LLM köré épített rendszer, amely célokat bont lépésekre, eszközöket hívhat, és visszacsatolások alapján halad. MCP: szabványosított mód arra, hogy AI alkalmazások külső adatokhoz, toolokhoz és workflow-khoz kapcsolódjanak.

How would you prototype an AI-assisted workflow?

I would start with a narrow business use case, define the input and expected output, then build a small prototype with clear evaluation examples. I would not start with a huge autonomous agent. I would first prove that the workflow saves time or improves quality.

What are the main risks with LLM features?

Hallucination, prompt injection, data leakage, wrong tool usage, cost, latency, and unclear evaluation. I would mitigate these with constrained prompts, validation, permissions, human review for sensitive actions, logging, and measurable acceptance criteria.

What is MCP useful for?

MCP is useful when an AI application needs a standard way to access tools, resources, or prompts from external systems. Instead of writing a unique integration for every model/tool pair, the idea is to expose capabilities through a common protocol.

How would you make an AI agent safer?

Limit its permissions, keep a human approval step for risky actions, validate tool inputs and outputs, log decisions, provide rollback where possible, and avoid giving the model broad access to sensitive systems.

12. AI válaszvázak angolul

AI interest answer

I have been actively exploring AI-assisted development and automation. What interests me is not AI as a buzzword, but practical workflows: reducing repetitive work, improving internal tools, helping users find information faster, or prototyping business processes. For production systems I would be careful with privacy, permissions, evaluation, and human control.

MCP answer

My understanding of MCP is that it standardizes how AI applications connect to tools, resources, and prompts. For a business application, that could mean exposing internal APIs or workflows to an AI assistant in a controlled way. The important part is not only the integration, but also security: what data the agent can access, what actions it can take, and which actions require user approval.

Agent answer

I would not start with a fully autonomous agent. I would start with a narrow use case, maybe a semi-automated workflow where the system drafts or suggests an action, and a human confirms it. Once the process is measurable and safe, we could automate more steps.

13. Logistics / pricing / operations mindset

Nem kell domain expertnek lenned, de értsd a gondolkodást.

- Logisztikai rendszereknél fontos a pontosság: árak, időablakok, kapacitás, státuszok, partneradatok.
- Sok edge case lehet: késés, törlés, módosítás, duplikált kérés, külső rendszer hibája.
- A pricing jellegű rendszerekben auditálhatóság és magyarázhatóság is fontos lehet.
- Business stakeholderrel beszélve ne frameworkökről kezdj, hanem workflow-ról, kockázatról, mérhető eredményről.
- Kérdezz rá: milyen core applications vannak, mely folyamatokat automatizálnák, milyen adatforrásokból dolgoznak.

DOMAIN

How would you learn an unfamiliar business domain?

I would start by mapping the main user roles, workflows, entities, and failure cases. I would ask stakeholders to walk me through real examples, not only ideal cases. Then I would connect the domain concepts to the technical model and validate assumptions early.

TRADE-OFF

How do you balance business speed and technical quality?

I try to identify what must be robust now and what can be iterated later. For core workflows, I would not compromise on data correctness, security, or observability. For UI details or internal tooling, I may choose a pragmatic first version if it helps validate the business value.

14. PHP legacy plus

Nem fő stack, de plusz pont a JD szerint.

Nem kell PHP expertnek pozicionálnod magad, hacsak nem vagy az. Inkább mondd azt, hogy legacy rendszerek megértésében és integrációjában komfortos vagy.

How would you approach a legacy PHP system?

First I would understand its business role, data model, dependencies, and deployment process. I would add tests or characterization checks around risky behavior where possible. For changes, I would prefer small safe increments and clear integration boundaries.

How would you integrate new Node.js services with legacy PHP?

I would define stable API boundaries, clear ownership of data, authentication, error handling, and logging. I would avoid duplicating business rules in two places unless there is a migration plan.

How do you modernize legacy code without breaking business workflows?

Use incremental migration, monitoring, feature flags where useful, and compare old vs new behavior. The goal is not rewriting for its own sake, but reducing risk while keeping the business running.

15. Behavioral kérdések

Rövid STAR válaszokkal készülj.

Tell me about a time you worked with business stakeholders.

Use a story where you clarified requirements, translated them into technical work, delivered incrementally, and adjusted based on feedback. End with the business result, not only the code you wrote.

Tell me about a production issue.

Use a story with clear incident handling: impact, investigation, communication, fix, monitoring, and prevention. Mention logs/metrics/correlation IDs if relevant.

Tell me about a disagreement in technical design.

Show that you can compare trade-offs calmly. Explain the options, constraints, how you listened, and how the team reached a decision.

Tell me about learning a new technology.

AI/MCP or AWS is a good angle. Show curiosity, but also structure: small prototype, documentation, testing, and practical outcome.

Tell me about a time you improved a process.

Automation is a strong story. Focus on repetitive pain, simple tool/script, measurable improvement, and adoption by the team.

16. Mini system design gyakorlat

Valószínű, hogy nem nagy FAANG design, hanem praktikus feature design.

Példa feladat

Design a shipment pricing dashboard where users can search shipments, see calculated price suggestions, and trigger a recalculation workflow.

Válasz struktúra:

- Clarify users and workflow: who uses it, what decision do they make?
- Data model: shipment, customer, route, price rule, price suggestion, recalculation status.
- Frontend: filters, table, details panel, loading/error/empty states, optimistic UI only if safe.
- Backend: REST endpoints, validation, authorization, pagination, audit log.
- Async work: recalculation can go through queue/job if slow.
- AWS: API + service + DB + queue + logs/metrics.
- Reliability: idempotent recalculation request, retry strategy, visible job status.
- Security: role-based access, no sensitive data in logs.
- Observability: correlation ID from UI to backend, metrics for failures and latency.

17. Kérdések, amiket te kérdezhetsz

Ezek nem túl agresszívek, de szakmainak hatnak.

- Which core logistics applications would this role work on first?
- How is the team split between frontend, backend, product, and business stakeholders?
- What parts of the stack are already on AWS, and what is still evolving?
- How mature is the CI/CD and release process today?
- What are the main pain points in the current platform: scalability, maintainability, speed of delivery, or business workflow complexity?
- How are AI agents or MCP being explored at the moment: internal tooling, customer-facing workflows, or developer productivity?
- What would success look like for this role in the first 3 to 6 months?
- Is PHP mainly for maintenance, integration, or migration from legacy systems?

Jó záró kérdés

Based on our conversation, is there any area where you would like me to clarify my experience or go deeper technically?

18. Kerülendő hibák

Ezek ronthatják az összképet.

- Ne mondd, hogy 'csak frontend' vagy, mert a JD full-stack és backend services irányt is kér.
- Ne buzzwordözz AI-ról. Mindig köss hozzá konkrét use case-t, kockázatot és kontrollt.
- Ne menj túl mély AWS architect válaszokba, ha nem kérik. Full-stack production awareness kell.
- Ne kritizáld a legacy PHP-t. Inkább mondd: business continuity, safe integration, incremental modernization.
- Ne csak technológiát sorolj. Minden válaszban legyen business outcome vagy trade-off.
- Ne ígérj expert szintet olyanban, amiben inkább learning/interest szinten vagy. A kíváncsiság itt is érték.

Biztonságos mondat, ha valamiben nem vagy expert

I have not used that specific tool deeply in production yet, but I understand the concept and I have worked with similar patterns. I would be comfortable learning it quickly, especially by starting with a small practical prototype.

19. Utolsó napi cheat sheet

A legfontosabb rövid válaszmagok.

React

Component composition, typed props, explicit loading/error states, avoid unnecessary global state.

TypeScript

Contracts, DTOs, discriminated unions, safer refactoring, not a replacement for tests.

Node.js

Non-blocking I/O, async error handling, validation, API boundaries, observability.

REST

Resources, methods, status codes, versioning, idempotency, consistent errors.

AWS

Where it runs, where data lives, logs/metrics, IAM least privilege, deployment/rollback.

CI/CD

Reproducible build, automated checks, staged deploy, rollback, monitoring.

AI/MCP

Narrow use case, controlled tool access, human approval, privacy/security, measurable value.

Business

Clarify workflow, edge cases, users, success metric, trade-offs.

20. Gyakorló kérdésbank

Válaszolj hangosan 1-2 percben.

- Walk me through a full-stack feature you built end-to-end.
- How do you decide what belongs on the frontend and what belongs on the backend?
- How do you handle business rules that change often?
- How do you make a React application maintainable as it grows?
- How do you design an API that multiple clients can safely use?
- How do you handle failures from an external API?
- How do you investigate a slow page or slow endpoint?
- How do you deploy safely?
- What AWS services have you used or studied, and how would you apply them here?
- What is your view on AI agents in business applications?
- How would you expose a business workflow to an AI assistant safely?
- How do you work with non-technical stakeholders?
- What do you do when requirements are unclear?
- What would you improve in a legacy system first?
- Why are you interested in this role?

21. Forrás és scope

A dokumentum a feltöltött Redspher Full Stack Developer job description alapján készült. A JD fő elemei: Contern, Luxembourg; business-oriented full-stack fejlesztés; React + TypeScript; backend services; AWS; REST APIs; Git/CI/CD; AI-driven solutions, MCP, AI agents, orchestration; automation; legacy PHP mint plusz.

Az MCP rövid magyarázata az official Model Context Protocol specifikáció általános leírására épül: MCP standardizált kapcsolódást ad LLM alkalmazások, külső adatforrások és toolok között; a specifikáció kiemeli a resources, prompts, tools és security/trust szempontokat.

Záró emlékeztető

A legjobb interjúválasz nem a leghosszabb. Legyen konkrét, üzletileg értelmezhető, és mutassa, hogy tudsz biztonságosan, pragmatikusan szállítani.